RANDOM FOREST (CLASSIFICATION)

In [1]:
```
# IMPORTING LIBRARIES
import numpy as np
import pandas as pd
import matplotlib.pyplot as mlt
import warnings
warnings.filterwarnings("ignore")
```

In [2]:
```
#LOADING DATASET
from sklearn.datasets import load_breast_cancer
data=load_breast_cancer()
data
```

Out[2]:
```
{'data': array([[1.799e+01, 1.038e+01, 1.228e+02, ..., 2.654e-01, 4.601e-01,
         1.189e-01],
        [2.057e+01, 1.777e+01, 1.329e+02, ..., 1.860e-01, 2.750e-01,
         8.902e-02],
        [1.969e+01, 2.125e+01, 1.300e+02, ..., 2.430e-01, 3.613e-01,
         8.758e-02],
        ...,
        [1.660e+01, 2.808e+01, 1.083e+02, ..., 1.418e-01, 2.218e-01,
         7.820e-02],
        [2.060e+01, 2.933e+01, 1.401e+02, ..., 2.650e-01, 4.087e-01,
         1.240e-01],
        [7.760e+00, 2.454e+01, 4.792e+01, ..., 0.000e+00, 2.871e-01,
         7.039e-02]], shape=(569, 30)),
 'target': array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1,
1,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0,
        0, 0, 1, 0, 1, 1, 1, 1, 1, 0, 0, 1, 0, 0, 1, 1, 1, 1, 0, 1, 0, 0,
        1, 1, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0, 0, 0,
        1, 1, 1, 0, 1, 1, 0, 0, 1, 1, 1, 0, 0, 1, 1, 1, 1, 0, 1, 1, 0, 1,
        1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0, 1, 0,
        0, 1, 0, 0, 1, 1, 0, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1,
        1, 1, 0, 1, 1, 1, 1, 0, 0, 1, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 1,
        1, 0, 1, 1, 0, 0, 0, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 0, 0, 1, 0, 0,
        0, 0, 1, 0, 0, 0, 1, 0, 1, 0, 1, 1, 0, 1, 0, 0, 0, 0, 1, 1, 0, 0,
        1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0, 1, 1, 0, 1, 1, 0, 0, 1, 0, 1, 1,
        1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 1, 1, 1, 1, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 0, 1, 0, 0, 1, 1,
        1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 0, 1, 0, 1, 1, 1, 1, 1,
        1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 0, 1, 1, 1, 1, 0, 0,
        0, 1, 1, 1, 1, 0, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 0,
        0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0,
        1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 1, 0, 0, 1, 1,
        1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 1, 0,
        1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 0, 0, 1, 0, 1, 1, 1, 1,
        1, 0, 1, 1, 0, 1, 0, 1, 1, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0,
        1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1,
        1, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0, 1, 0, 1, 0, 1, 1,
        1, 1, 1, 0, 1, 1, 0, 1, 0, 1, 0, 0, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1,
        1, 1, 1, 1, 1, 0, 1, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
        1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 1]),
 'frame': None,
 'target_names': array(['malignant', 'benign'], dtype='<U9'),
 'DESCR': '.. _breast_cancer_dataset:\n\nBreast cancer Wisconsin (diagnostic) data
set\n--------------------------------------------\n\n**Data Set Characteristics:**
\n\n:Number of Instances: 569\n\n:Number of Attributes: 30 numeric, predictive att
ributes and the class\n\n:Attribute Information:\n    - radius (mean of distances
from center to points on the perimeter)\n    - texture (standard deviation of gray
-scale values)\n    - perimeter\n    - area\n    - smoothness (local variation in
radius lengths)\n    - compactness (perimeter^2 / area - 1.0)\n    - concavity (se
verity of concave portions of the contour)\n    - concave points (number of concav
e portions of the contour)\n    - symmetry\n    - fractal dimension ("coastline ap
proximation" - 1)\n\n    The mean, standard error, and "worst" or largest (mean of
the three\n    worst/largest values) of these features were computed for each imag
e,\n    resulting in 30 features.  For instance, field 0 is Mean Radius, field\n
10 is Radius SE, field 20 is Worst Radius.\n\n    - class:\n            - WDBC-Mal
ignant\n            - WDBC-Benign\n\n:Summary Statistics:\n\n====================
```

```
================ ====== ======\n                                        Min    Max
\n===================================== ====== ======\nradius (mean):
6.981  28.11\ntexture (mean):                       9.71   39.28\nperimeter (mea
n):                    43.79  188.5\narea (mean):                          143.5
2501.0\nsmoothness (mean):                   0.053  0.163\ncompactness (mean):
0.019  0.345\nconcavity (mean):                     0.0    0.427\nconcave points
(mean):               0.0    0.201\nsymmetry (mean):                     0.106
0.304\nfractal dimension (mean):            0.05   0.097\nradius (standard erro
r):             0.112  2.873\ntexture (standard error):            0.36   4.885
\nperimeter (standard error):          0.757  21.98\narea (standard error):
6.802  542.2\nsmoothness (standard error):          0.002  0.031\ncompactness (sta
ndard error):         0.002  0.135\nconcavity (standard error):          0.0
0.396\nconcave points (standard error):      0.0    0.053\nsymmetry (standard erro
r):            0.008  0.079\nfractal dimension (standard error):   0.001  0.03\nra
dius (worst):                        7.93   36.04\ntexture (worst):
12.02  49.54\nperimeter (worst):                    50.41  251.2\narea (worst):
185.2  4254.0\nsmoothness (worst):                   0.071  0.223\ncompactness (wo
rst):                  0.027  1.058\nconcavity (worst):                    0.0
1.252\nconcave points (worst):               0.0    0.291\nsymmetry (worst):
0.156  0.664\nfractal dimension (worst):            0.055  0.208\n================
==================== ====== ======\n\n:Missing Attribute Values: None\n\n:Class D
istribution: 212 - Malignant, 357 - Benign\n\n:Creator:  Dr. William H. Wolberg,
W. Nick Street, Olvi L. Mangasarian\n\n:Donor: Nick Street\n\n:Date: November, 199
5\n\nThis is a copy of UCI ML Breast Cancer Wisconsin (Diagnostic) datasets.\nhttp
s://goo.gl/U2Uwz2\n\nFeatures are computed from a digitized image of a fine needle
\naspirate (FNA) of a breast mass.  They describe\ncharacteristics of the cell nuc
lei present in the image.\n\nSeparating plane described above was obtained using\n
Multisurface Method-Tree (MSM-T) [K. P. Bennett, "Decision Tree\nConstruction Via
Linear Programming." Proceedings of the 4th\nMidwest Artificial Intelligence and C
ognitive Science Society,\npp. 97-101, 1992], a classification method which uses l
inear\nprogramming to construct a decision tree.  Relevant features\nwere selected
using an exhaustive search in the space of 1-4\nfeatures and 1-3 separating plane
s.\n\nThe actual linear program used to obtain the separating plane\nin the 3-dime
nsional space is that described in:\n[K. P. Bennett and O. L. Mangasarian: "Robust
Linear\nProgramming Discrimination of Two Linearly Inseparable Sets",\nOptimizatio
n Methods and Software 1, 1992, 23-34].\n\nThis database is also available through
the UW CS ftp server:\n\nftp ftp.cs.wisc.edu\ncd math-prog/cpo-dataset/machine-lea
rn/WDBC/\n\n.. dropdown:: References\n\n  - W.N. Street, W.H. Wolberg and O.L. Man
gasarian. Nuclear feature extraction\n    for breast tumor diagnosis. IS&T/SPIE 19
93 International Symposium on\n    Electronic Imaging: Science and Technology, vol
ume 1905, pages 861-870,\n    San Jose, CA, 1993.\n  - O.L. Mangasarian, W.N. Stre
et and W.H. Wolberg. Breast cancer diagnosis and\n    prognosis via linear program
ming. Operations Research, 43(4), pages 570-577,\n    July-August 1995.\n  - W.H.
Wolberg, W.N. Street, and O.L. Mangasarian. Machine learning techniques\n    to di
agnose breast cancer from fine-needle aspirates. Cancer Letters 77 (1994)\n    163
-171.\n',
 'feature_names': array(['mean radius', 'mean texture', 'mean perimeter', 'mean ar
ea',
        'mean smoothness', 'mean compactness', 'mean concavity',
        'mean concave points', 'mean symmetry', 'mean fractal dimension',
        'radius error', 'texture error', 'perimeter error', 'area error',
        'smoothness error', 'compactness error', 'concavity error',
        'concave points error', 'symmetry error',
        'fractal dimension error', 'worst radius', 'worst texture',
        'worst perimeter', 'worst area', 'worst smoothness',
        'worst compactness', 'worst concavity', 'worst concave points',
```

```
       'worst symmetry', 'worst fractal dimension'], dtype='<U23'),
 'filename': 'breast_cancer.csv',
 'data_module': 'sklearn.datasets.data'}
```

In [4]:
```
#CONVERTING DATA INTO A DATA FRAME
df=pd.DataFrame(np.c_[data.data,data.target],
                columns=[list(data.feature_names)+["target"]])
df
```

Out[4]:

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	sy
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.30010	0.14710	
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.08690	0.07017	
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.19740	0.12790	
3	11.42	20.38	77.58	386.1	0.14250	0.28390	0.24140	0.10520	
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.19800	0.10430	
...	...	...	...	...	...	...	...	...	
564	21.56	22.39	142.00	1479.0	0.11100	0.11590	0.24390	0.13890	
565	20.13	28.25	131.20	1261.0	0.09780	0.10340	0.14400	0.09791	
566	16.60	28.08	108.30	858.1	0.08455	0.10230	0.09251	0.05302	
567	20.60	29.33	140.10	1265.0	0.11780	0.27700	0.35140	0.15200	
568	7.76	24.54	47.92	181.0	0.05263	0.04362	0.00000	0.00000	

569 rows × 31 columns

In [5]:
```
# RANDOM FOREST (CLASSIFICATION)
# STEP 1: ASSIGNING VARIABLES(INDEPENDENT/DEPENDENT)

# INDEPENDENT VARIABLE
x=df.drop("target",axis=1)

# DEPENDENT VARIABLE
y=df[["target"]]
```

In [8]:
```
# STEP 2:IMPORT TRAIN TEST SPLIT
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test,=train_test_split(x,y,test_size=0.3,random_state=42)
```

In [9]:
```
print("Original size : ",df.shape)
print("x_train size : ",x_train.shape)
print("x_test size : ",x_test.shape)
print("y_train size : ",y_train.shape)
print("y_test size : ",y_test.shape)
```

```
Original size :  (569, 31)
x_train size :  (398, 30)
x_test size :  (171, 30)
y_train size :  (398, 1)
y_test size :  (171, 1)
```

In [10]:
```
# STEP 3: APPLY RANDOM FOREST CLASSIFICATION
from sklearn.ensemble import RandomForestClassifier
classifier=RandomForestClassifier(n_estimators=5)
classifier
```

Out[10]:
```
RandomForestClassifier
▶ Parameters
```

In [12]:
```
# STEP 4 :FIT TO DATA
# Use only train(x_train,y_train)
classifier.fit(x_train,y_train)
```

Out[12]:
```
RandomForestClassifier
▶ Parameters
```

In [14]:
```
# CHECK ACCURACY
classifier.score(x_test,y_test)
```

Out[14]:
```
0.9649122807017544
```

In [15]:
```
# PREDICTION OF NEW DATA
# IMPORT NUMPY FIRST(DATA MUST IN ARRAY FORM)
patient=np.array([[21.56,22.39,142.00,1479.0,0.11100,0.11590,0.24390,0.13890,0.1726
                   26.40,26.40,26.40,26.40,26.40,26.40,26.40,26.40,26.40,26.40,
                   26.40,26.40,166.10,2027.0,0.14100,0.21130,0.4107,0.2216,0.2060,0
patient
```

Out[15]:
```
array([[2.156e+01, 2.239e+01, 1.420e+02, 1.479e+03, 1.110e-01, 1.159e-01,
        2.439e-01, 1.389e-01, 1.726e-01, 5.623e-02, 2.640e+01, 2.640e+01,
        2.640e+01, 2.640e+01, 2.640e+01, 2.640e+01, 2.640e+01, 2.640e+01,
        2.640e+01, 2.640e+01, 2.640e+01, 2.640e+01, 1.661e+02, 2.027e+03,
        1.410e-01, 2.113e-01, 4.107e-01, 2.216e-01, 2.060e-01, 7.115e-02]])
```

In [16]:
```
prediction=classifier.predict(patient)
if (prediction[0]==0):
    print("Patient has Malignent tumor")
else:
    print("Patient has Benign tumor")
```

```
Patient has Malignent tumor
```

In [17]:
```
# EVALUATE MODEL
from sklearn.metrics import(accuracy_score,classification_report,confusion_matrix,C
```

In [19]:
```
y_pred=classifier.predict(x_test)
print("accuracy:",accuracy_score(y_test,y_pred))
print("\nclassification Report:\n",classification_report(y_test,y_pred))
```

```
accuracy: 0.9649122807017544

classification Report:
               precision    recall  f1-score   support

         0.0       0.93      0.98      0.95        63
         1.0       0.99      0.95      0.97       108

    accuracy                           0.96       171
   macro avg       0.96      0.97      0.96       171
weighted avg       0.97      0.96      0.97       171
```

In [21]:
```
cm = confusion_matrix(y_test, y_pred)

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=data.target_names
)
disp.plot(cmap="Blues")
mlt.title("Random Forest Classification")
mlt.show()
```

